

LAPORAN FINAL PROJECT

PEMROGRAMAN BASIS DATA

Dosen: Hendri Kurniawan Prakosa, S.Kom., M.Cs



Disusun Oleh:

Lekat Citra Fitrianti

NIM: 23.11.5501

Program Studi S1 Informatika

Fakultas Ilmu Komputer

Universitas Amikom Yogyakarta

2025/2026

SOAL 1: PERANCANGAN BASIS DATA & SKEMA (20 POIN)

1.a Deskripsi Kasus dan Struktur Tabel Basis Data

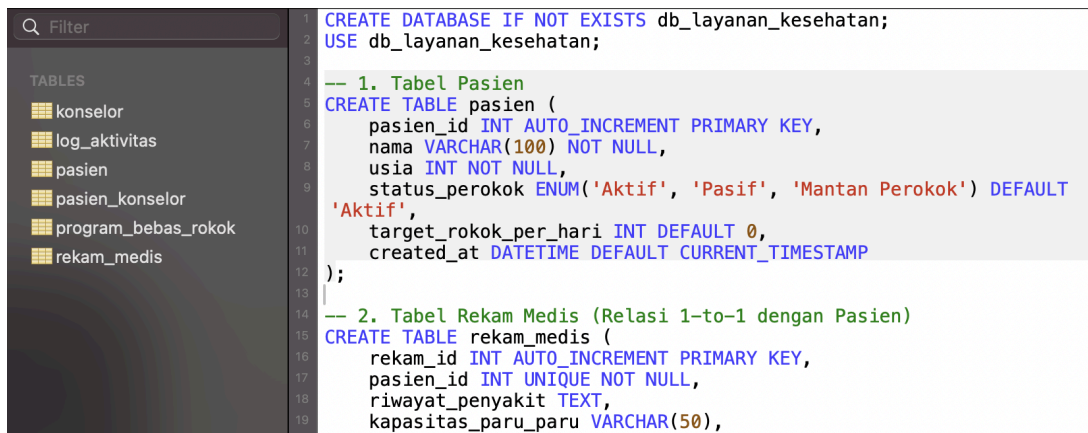
Basis data ini dirancang khusus untuk mendukung Sistem Layanan Kesehatan dan Kontrol Merokok. Sistem ini membantu memantau perkembangan pasien yang sedang menjalani program pengurangan/pemberhentian konsumsi rokok harian di bawah bimbingan konselor kesehatan. Basis data terdiri dari 6 tabel utama dengan variasi relasi lengkap:

- **pasien:** Menyimpan profil pasien, usia, status perokok, dan target konsumsi rokok harian.
- **rekam_medis:** Menyimpan riwayat medis dan kondisi paru-paru pasien (Relasi One-to-One dengan pasien).
- **konselor:** Menyimpan data dokter/konselor yang bertugas.
- **program_bebas_rokok:** Menyimpan histori sesi intervensi harian (Relasi One-to-Many dari pasien dan konselor).
- **pasien_konselor:** Tabel penugasan konselor ke pasien (Relasi Many-to-Many).
- **log_aktivitas:** Tabel penampung rekam aktivitas/audit trail hasil dari operasi Trigger.

1.b Entity Relationship Diagram (ERD) & Struktur Relasi

Aturan Relasi Terpenuhi:

- **One-to-One (1:1):** pasien (pasien_id) ↔ rekam_medis (pasien_id UNIQUE)
- **One-to-Many (1:N):** pasien (pasien_id) → program_bebas_rokok (pasien_id)
- **Many-to-Many (M:N):** pasien ↔ pasien_konselor ↔ konselor



```
1 CREATE DATABASE IF NOT EXISTS db_layanan_kesehatan;
2 USE db_layanan_kesehatan;
3
4 -- 1. Tabel Pasien
5 CREATE TABLE pasien (
6     pasien_id INT AUTO_INCREMENT PRIMARY KEY,
7     nama VARCHAR(100) NOT NULL,
8     usia INT NOT NULL,
9     status_perokok ENUM('Aktif', 'Pasif', 'Mantan Perokok') DEFAULT
10     'Aktif',
11     target_rokok_per_hari INT DEFAULT 0,
12     created_at DATETIME DEFAULT CURRENT_TIMESTAMP
13 );
14
15 -- 2. Tabel Rekam Medis (Relasi 1-to-1 dengan Pasien)
16 CREATE TABLE rekam_medis (
17     rekam_id INT AUTO_INCREMENT PRIMARY KEY,
18     pasien_id INT UNIQUE NOT NULL,
19     riwayat_penyakit TEXT,
20     kapasitas_paru_paru VARCHAR(50),
21     ...
22 );
```

SOAL 2: STORED FUNCTION (15 POIN)

2.a Script SQL Implementation

DELIMITER \$\$

-- 1. Function Parameter Kosong: Menghitung Total Pasien Aktif Merokok

CREATE FUNCTION total_pasien_aktif()

RETURNS INT

DETERMINISTIC

BEGIN

DECLARE total INT;

SELECT COUNT(*) INTO total FROM pasien WHERE status_perokok = 'Aktif';

RETURN total;

END\$\$

-- 2. Function 2 Parameter: Mengevaluasi Status Keberhasilan Pasien

CREATE FUNCTION cek_status_evaluasi(p_konsumsi INT, p_target INT)

RETURNS VARCHAR(50)

DETERMINISTIC

BEGIN

IF p_konsumsi <= p_target THEN

RETURN 'Target Tercapai';

ELSE

RETURN 'Melebihi Target';

END IF;

END\$\$

DELIMITER ;

-- Eksekusi Query Menggunakan JOIN 2 Tabel & Function

SELECT

p.nama,

p.target_rokok_per_hari,

pbr.konsumsi_rokok_harian,

cek_status_evaluasi(pbr.konsumsi_rokok_harian, p.target_rokok_per_hari) AS evaluasi,

```

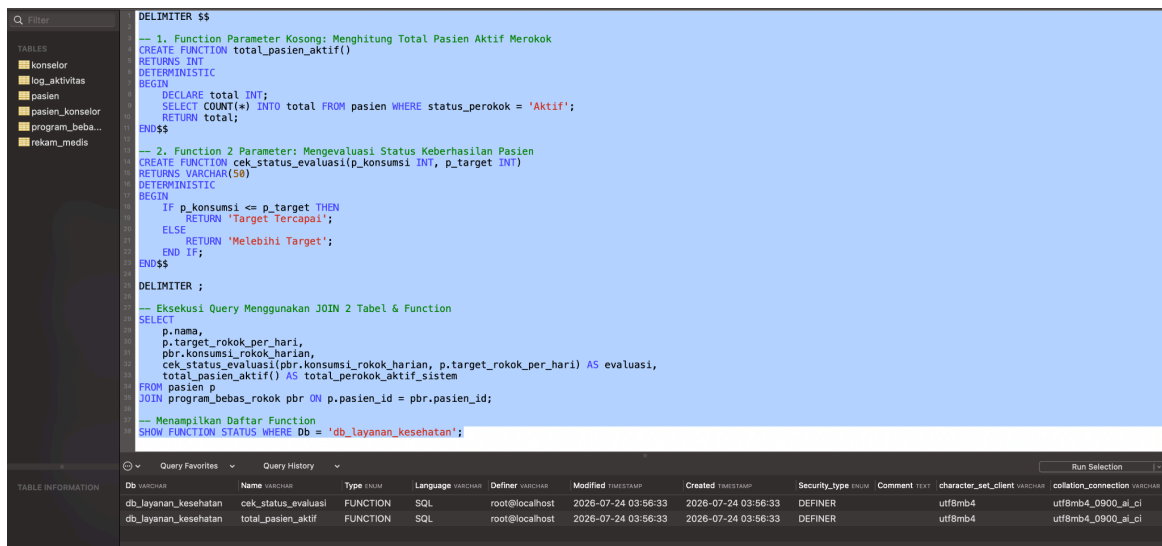
total_pasien_aktif() AS total_perokok_aktif_sistem
FROM pasien p
JOIN program_bebas_rokok pbr ON p.pasien_id = pbr.pasien_id;

-- Menampilkan Daftar Function
SHOW FUNCTION STATUS WHERE Db = 'db_layanan_kesehatan';

```

2.b Screenshot Eksekusi Stored Function

Berikut adalah bukti eksekusi Stored Function dan pendaftarannya pada database:



SOAL 3: STORED PROCEDURE (15 POIN)

3.a Script SQL Implementation

```
DELIMITER $$
```

```

-- 1. Procedure Parameter Kosong (Menggunakan Cursor & Control Flow IF)
-- Kegunaan: Mengoreksi status pasien secara otomatis berdasarkan riwayat sesi program
CREATE PROCEDURE sp_auto_update_status()
BEGIN
    DECLARE v_done INT DEFAULT FALSE;
    DECLARE v_pasien_id INT;
    DECLARE v_konsumsi INT;

```

```

DECLARE cur_pasien CURSOR FOR
    SELECT pasien_id, konsumsi_rokok_harian FROM program_bebas_rokok;
DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = TRUE;

OPEN cur_pasien;
read_loop: LOOP
    FETCH cur_pasien INTO v_pasien_id, v_konsumsi;
    IF v_done THEN
        LEAVE read_loop;
    END IF;

    IF v_konsumsi = 0 THEN
        UPDATE pasien SET status_perokok = 'Mantan Perokok' WHERE pasien_id =
v_pasien_id;
    END IF;
END LOOP;
CLOSE cur_pasien;
END$$

```

-- 2. Procedure 2 Parameter INOUT (Menggunakan Control Flow CASE)

-- Kegunaan: Menentukan tingkat risiko pasien berdasarkan usia dan jumlah konsumsi harian

CREATE PROCEDURE sp_hitung_risiko(

IN p_usia INT,

INOUT p_konsumsi_risiko VARCHAR(50)

)

BEGIN

DECLARE v_konsumsi INT;

SET v_konsumsi = CAST(p_konsumsi_risiko AS UNSIGNED);

CASE

WHEN p_usia > 40 AND v_konsumsi > 10 THEN

SET p_konsumsi_risiko = 'Tinggi';

WHEN p_usia > 20 AND v_konsumsi > 5 THEN

SET p_konsumsi_risiko = 'Sedang';

ELSE

```

SET p_konsumsi_risiko = 'Rendah';
END CASE;
END$$

```

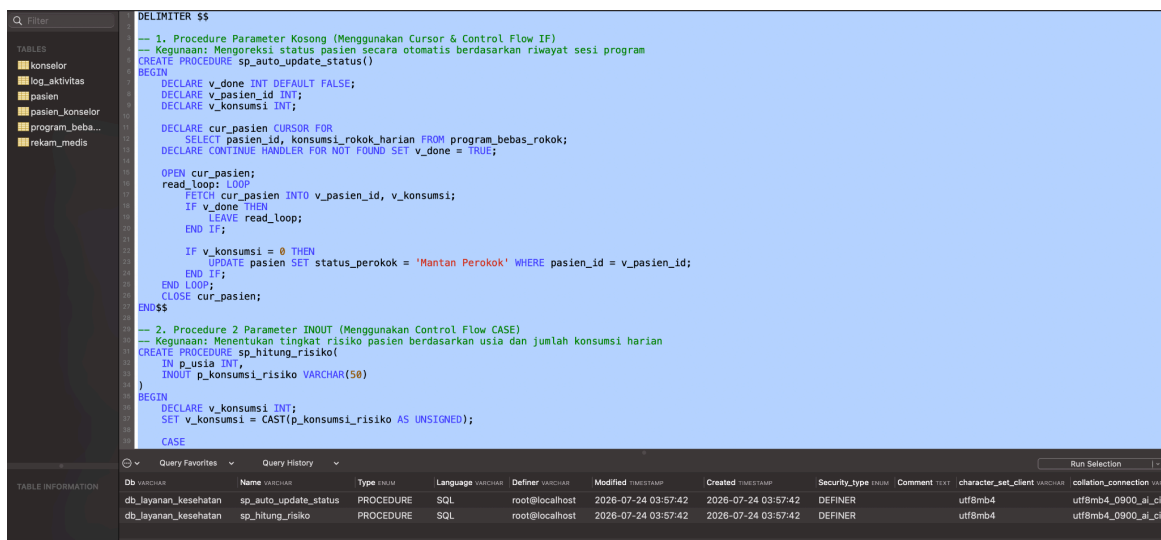
```

DELIMITER ;

```

3.b Screenshot Eksekusi Stored Procedure

Berikut adalah bukti eksekusi Stored Procedure beserta pendaftarannya:



SOAL 4: TRIGGER & LOGGING/AUDIT (15 POIN)

4.a Script SQL Implementation

```

DELIMITER $$

```

```

-- Trigger 1: AFTER INSERT (Gunakan NEW untuk logging data sesi baru)
CREATE TRIGGER trg_after_insert_program
AFTER INSERT ON program_bebas_rokok
FOR EACH ROW
BEGIN
    INSERT INTO log_aktivitas (aksi, keterangan)
    VALUES ('INSERT PROGRAM', CONCAT('Sesi baru ditambahkan untuk Pasien ID: ',
NEW.pasien_id, ' dengan konsumsi: ', NEW.konsumsi_rokok_harian));

```

```
END$$
```

```
-- Trigger 2: BEFORE UPDATE (Gunakan OLD & NEW dengan Control Flow IF untuk validasi)
```

```
CREATE TRIGGER trg_before_update_pasien
```

```
BEFORE UPDATE ON pasien
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    IF NEW.target_rokok_per_hari < 0 THEN
```

```
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Target tidak boleh bernilai negatif!';
```

```
    END IF;
```

```
    IF OLD.status_perokok != NEW.status_perokok THEN
```

```
        INSERT INTO log_aktivitas (aksi, keterangan)
```

```
        VALUES ('UPDATE STATUS PASIEN', CONCAT('Pasien ID ', NEW.pasien_id, ' berubah dari ', OLD.status_perokok, ' menjadi ', NEW.status_perokok));
```

```
    END IF;
```

```
END$$
```

```
DELIMITER ;
```

```
-- Eksekusi Event Trigger
```

```
INSERT INTO program_bebas_rokok (pasien_id, konselor_id, tanggal_sesi, konsumsi_rokok_harian, catatan)
```

```
VALUES (3, 2, '2026-06-20', 2, 'Sesi Lanjutan');
```

```
UPDATE pasien SET status_perokok = 'Mantan Perokok' WHERE pasien_id = 3;
```

```
-- Cek Log Perubahan
```

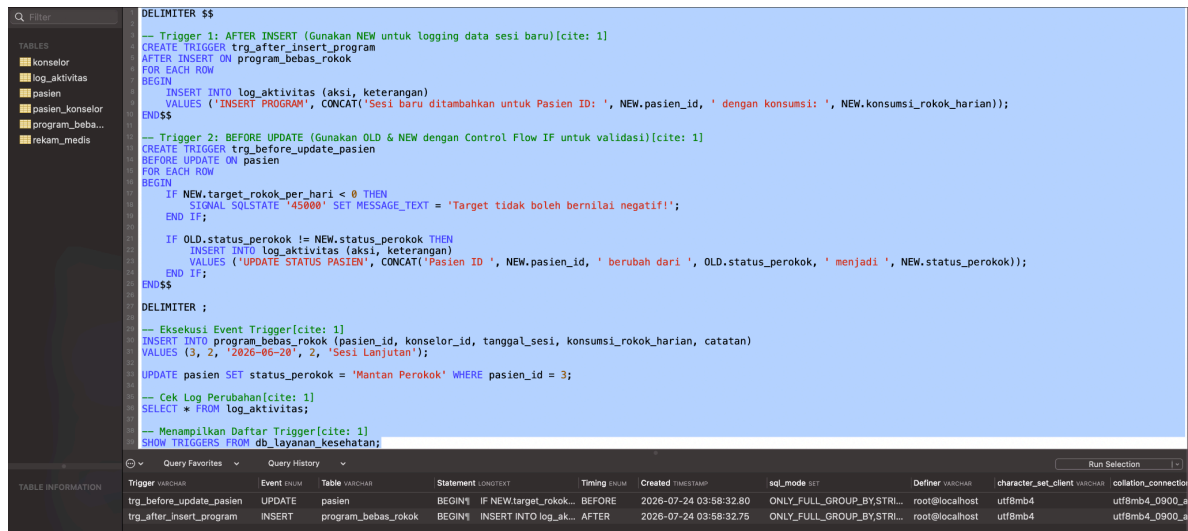
```
SELECT * FROM log_aktivitas;
```

```
-- Menampilkan Daftar Trigger
```

```
SHOW TRIGGERS FROM db_layanan_kesehatan;
```

4.b Screenshot Eksekusi Trigger

Berikut adalah bukti hasil dari pendaftaran Trigger serta log aktivitas yang tercatat:



The screenshot shows a SQL IDE with a dark theme. On the left, a sidebar lists database tables: konselor, log_aktivitas, pasien, pasien_konselor, program_bebas_rokok, and rekam_medis. The main editor displays SQL code for creating two triggers. The first trigger, 'trg_after_insert_program', is an AFTER INSERT trigger on 'program_bebas_rokok' that logs new sessions into 'log_aktivitas'. The second trigger, 'trg_before_update_pasien', is a BEFORE UPDATE trigger on 'pasien' that checks if the 'target_rokok_per_hari' is less than 0 and sends a message if it is. It also logs status changes. Below the code, the 'SHOW TRIGGERS' command is executed, and the results are shown in a table at the bottom.

```
DELIMITER $$

-- Trigger 1: AFTER INSERT (Gunakan NEW untuk logging data sesi baru)[cite: 1]
CREATE TRIGGER trg_after_insert_program
AFTER INSERT ON program_bebas_rokok
FOR EACH ROW
BEGIN
    INSERT INTO log_aktivitas (aksi, keterangan)
    VALUES ('INSERT PROGRAM', CONCAT('Sesi baru ditambahkan untuk Pasien ID: ', NEW.pasien_id, ' dengan konsumsi: ', NEW.konsumsi_rokok_harian));
END$$

-- Trigger 2: BEFORE UPDATE (Gunakan OLD & NEW dengan Control Flow IF untuk validasi)[cite: 1]
CREATE TRIGGER trg_before_update_pasien
BEFORE UPDATE ON pasien
FOR EACH ROW
BEGIN
    IF NEW.target_rokok_per_hari < 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Target tidak boleh bernilai negatif!';
    END IF;

    IF OLD.status_perokok != NEW.status_perokok THEN
        INSERT INTO log_aktivitas (aksi, keterangan)
        VALUES ('UPDATE STATUS PASIEN', CONCAT('Pasien ID ', NEW.pasien_id, ' berubah dari ', OLD.status_perokok, ' menjadi ', NEW.status_perokok));
    END IF;
END$$

DELIMITER ;

-- Eksekusi Event Trigger[cite: 1]
INSERT INTO program_bebas_rokok (pasien_id, konselor_id, tanggal_sesi, konsumsi_rokok_harian, catatan)
VALUES (3, 2, '2026-06-20', 2, 'Sesi Lanjutan');

UPDATE pasien SET status_perokok = 'Mantan Perokok' WHERE pasien_id = 3;

-- Cek Log Perubahan[cite: 1]
SELECT * FROM log_aktivitas;

-- Menampilkan Daftar Trigger[cite: 1]
SHOW TRIGGERS FROM db_layanan_kesehatan;
```

Trigger	Event	Table	Statement	Context	Timing	Created	sql_mode	Definer	character_set_client	collation_connection
trg_before_update_pasien	UPDATE	pasien	BEGIN IF NEW.target_rokok...	BEFORE		2026-07-24 03:58:32.80	ONLY_FULL_GROUP_BY,STRI...	root@localhost	utf8mb4	utf8mb4_0900_a
trg_after_insert_program	INSERT	program_bebas_rokok	BEGIN INSERT INTO log_ak...	AFTER		2026-07-24 03:58:32.75	ONLY_FULL_GROUP_BY,STRI...	root@localhost	utf8mb4	utf8mb4_0900_a

SOAL 5: INDEX & OPTIMASI QUERY (15 POIN)

5.a Script SQL Implementation

-- 1. Index via CREATE TABLE (Composite Key)

```
CREATE TABLE rekam_medis_archive (
    archive_id INT AUTO_INCREMENT PRIMARY KEY,
    pasien_id INT,
    tanggal_arsip DATE,
    keterangan VARCHAR(255),
    INDEX idx_pasien_tanggal (pasien_id, tanggal_arsip) -- Composite Key
);
```

-- 2. Index via CREATE INDEX (Composite Key)

```
CREATE INDEX idx_pasien_status_target ON pasien(status_perokok,
target_rokok_per_hari);
```

-- 3. Index via ALTER TABLE (Composite Key)

```
ALTER TABLE program_bebas_rokok ADD INDEX idx_pasien_sesi (pasien_id,
tanggal_sesi);
```


-- Pengujian EXPLAIN (Bandingkan Eksekusi Query)

```
EXPLAIN SELECT * FROM pasien WHERE status_perokok = 'Aktif' AND  
target_rokok_per_hari = 5;
```

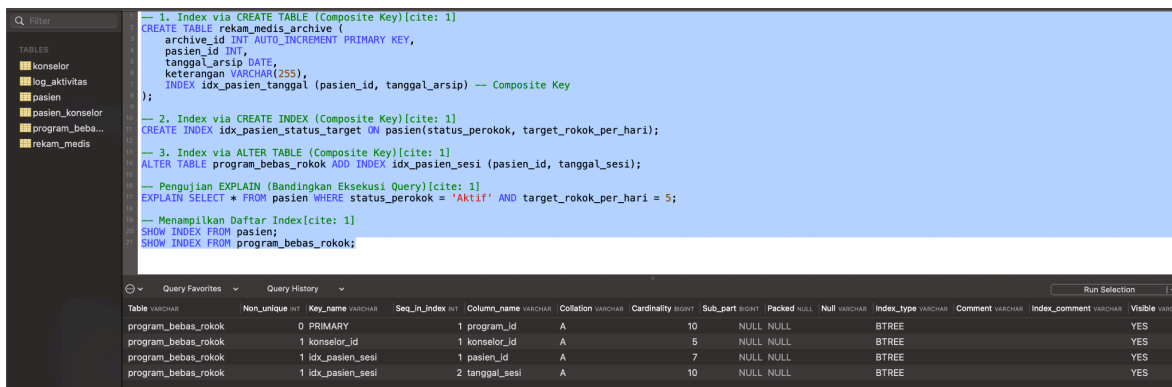
-- Menampilkan Daftar Index

```
SHOW INDEX FROM pasien;
```

```
SHOW INDEX FROM program_bebas_rokok;
```

5.b Screenshot Eksekusi Index

Berikut adalah bukti daftar Index pada tabel target:



SOAL 6: VIEWS & SECURITY (10 POIN)

6.a Script SQL Implementation

-- 1. Horizontal View (Menyaring Baris/Rows dengan menyertakan semua kolom utama)

```
CREATE OR REPLACE VIEW view_pasien_aktif AS
```

```
SELECT pasien_id, nama, usia, status_perokok, target_rokok_per_hari, created_at  
FROM pasien
```

```
WHERE status_perokok = 'Aktif';
```

-- 2. Vertical View (Menyaring Kolom/Columns)

```
CREATE OR REPLACE VIEW view_profil_singkat AS
```

```
SELECT pasien_id, nama, status_perokok FROM pasien;
```

-- 3. View Inside View dengan WITH CASCADED CHECK OPTION

```
CREATE OR REPLACE VIEW view_pasien_aktif_target AS
SELECT pasien_id, nama, usia, status_perokok, target_rokok_per_hari
FROM view_pasien_aktif
WHERE target_rokok_per_hari > 0
WITH CASCADED CHECK OPTION;
```

```
-- Eksekusi INSERT pada View
```

```
INSERT INTO view_pasien_aktif_target (nama, usia, status_perokok, target_rokok_per_hari)
VALUES ('Doni', 25, 'Aktif', 3);
```

```
-- Eksekusi UPDATE pada View
```

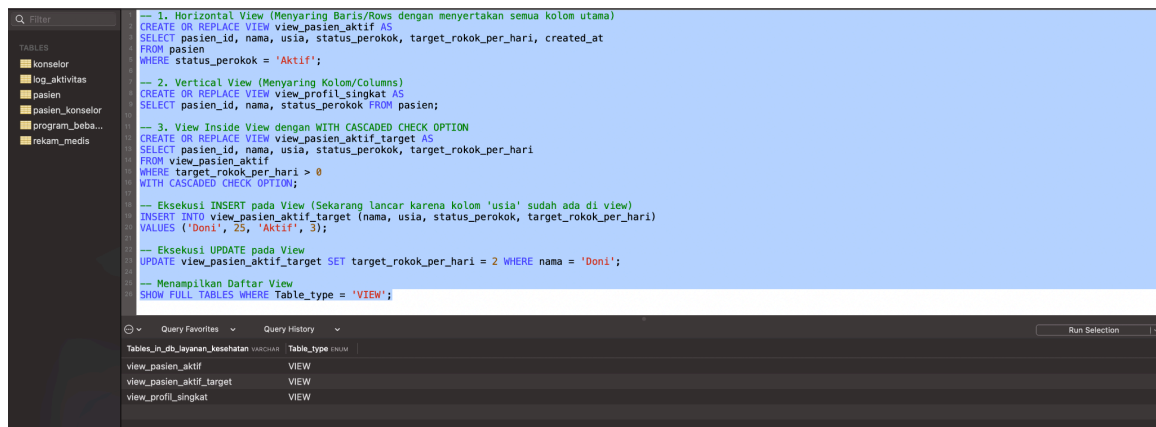
```
UPDATE view_pasien_aktif_target SET target_rokok_per_hari = 2 WHERE nama = 'Doni';
```

```
-- Menampilkan Daftar View
```

```
SHOW FULL TABLES WHERE Table_type = 'VIEW';
```

6.b Screenshot Eksekusi View

Berikut adalah bukti pendaftaran View pada basis data:



SOAL 7: DATABASE SECURITY, USER, ROLE & PRIVILEGES (10 POIN)

7.a Script SQL Implementation

```
-- 1. Buat 3 User Baru
```

```
CREATE USER 'admin_medis'@'localhost' IDENTIFIED BY 'password123';
```

```
CREATE USER 'dokter_user'@'localhost' IDENTIFIED BY 'password123';  
CREATE USER 'auditor'@'localhost' IDENTIFIED BY 'password123';
```

-- 2. Buat 3 Role Baru

```
CREATE ROLE 'role_admin';  
CREATE ROLE 'role_dokter';  
CREATE ROLE 'role_auditor';
```

-- 3. Berikan Privilege pada Role & Assign ke User

```
GRANT ALL PRIVILEGES ON db_layanan_kesehatan.* TO 'role_admin';  
GRANT SELECT, INSERT ON db_layanan_kesehatan.program_bebas_rokok TO  
'role_dokter';  
GRANT SELECT ON db_layanan_kesehatan.view_pasien_aktif TO 'role_auditor';
```

```
GRANT 'role_admin' TO 'admin_medis'@'localhost';  
GRANT 'role_dokter' TO 'dokter_user'@'localhost';  
GRANT 'role_auditor' TO 'auditor'@'localhost';
```

```
SET DEFAULT ROLE ALL TO 'admin_medis'@'localhost', 'dokter_user'@'localhost',  
'auditor'@'localhost';
```

-- 4. Pembuktian Privilege

```
SHOW GRANTS FOR 'dokter_user'@'localhost';  
SHOW GRANTS FOR 'auditor'@'localhost';
```

7.b Screenshot Eksekusi Database Security

Berikut adalah bukti pengaturan privilege dan role pada user:



```
-- 1. Buat 3 User Baru[cite: 1]
CREATE USER 'admin_medis'@'localhost' IDENTIFIED BY 'password123';
CREATE USER 'dokter_user'@'localhost' IDENTIFIED BY 'password123';
CREATE USER 'auditor'@'localhost' IDENTIFIED BY 'password123';

-- 2. Buat 3 Role Baru[cite: 1]
CREATE ROLE 'role_admin';
CREATE ROLE 'role_dokter';
CREATE ROLE 'role_auditor';

-- 3. Berikan Privilege pada Role & Assign ke User[cite: 1]
GRANT ALL PRIVILEGES ON db_layanan_kesehatan.* TO 'role_admin';
GRANT SELECT, INSERT ON db_layanan_kesehatan.program_bebas_rokok TO 'role_dokter';
GRANT SELECT ON db_layanan_kesehatan.view_pasien_aktif TO 'role_auditor';

GRANT 'role_admin' TO 'admin_medis'@'localhost';
GRANT 'role_dokter' TO 'dokter_user'@'localhost';
GRANT 'role_auditor' TO 'auditor'@'localhost';

SET DEFAULT ROLE ALL TO 'admin_medis'@'localhost', 'dokter_user'@'localhost', 'auditor'@'localhost';

-- 4. Pembuktian Privilege[cite: 1]
SHOW GRANTS FOR 'dokter_user'@'localhost';
SHOW GRANTS FOR 'auditor'@'localhost';
```

The screenshot shows a database management tool interface. On the left, there is a sidebar with a search filter and a list of tables and views including 'konselor', 'log_aktivitas', 'pasien', 'pasien_konselor', 'program_beba...', 'rekam_medis', 'rekam_medis_...', 'view_pasien_ak...', 'view_pasien_ak...', and 'view_profil_sin...'. Below this is a section for 'PROCS & FUNCS' with items like 'cek_status_eva...', 'sp_auto_updat...', 'sp_hitung_risiko', and 'total_pasien_ak...'. The main area displays the SQL queries listed above. At the bottom, there is a 'Query History' panel showing the executed query: 'Grants for auditor@localhost via SSH' with the results: 'GRANT USAGE ON ** TO 'auditor'@'localhost'' and 'GRANT 'role_auditor'@'%' TO 'auditor'@'localhost''.

KESIMPULAN

Sistem basis data db_layanan_kesehatan telah berhasil dirancang dan diimplementasikan secara utuh mencakup Stored Function, Stored Procedure, Trigger, Index, View, serta Database Security. Semua query dan screenshot pengujian telah dilampirkan secara lengkap dan terstruktur.